LECTURE 3

Type conversion, Char type, Logical expressions, Conditional Expressions,

Conditional execution: If-Else, Switch

RECAP

- Numeric DataTypes: `int`, `float`
  - Qualifiers:
  - Specifiers:
- Definition, printing, inputting
  -
  -
  -

RECAP

- Numeric DataTypes: `int`, `float`
  - Qualifiers: `long`, `double`, `unsigned`, `short`
  - Specifiers:
- Definition, printing, inputting
  -
  -
  -

RECAP

- Numeric DataTypes: `int`, `float`
  - Qualifiers: `long`, `double`, `unsigned`, `short`
  - Specifiers: `%d`, `%u`, `%f` (`l`,`u`,`h` modifiers)
- Definition, printing, inputting
  -
  -
  -

RECAP

- Numeric DataTypes: `int`, `float`
  - Qualifiers: `long`, `double`, `unsigned`, `short`
  - Specifiers: `%d`, `%u`, `%f` (`l`,`u`,`h` modifiers)
- Definition, printing, inputting
  - `int number;`
  - `printf("%d",number);`
  - `scanf("%d",&number);`

RECAP

- Math operators: + - * / % ( )
- Math Library:
  - Math Constants:
  - Elementary: ...
  - Tringonometric: ...
  - Special Functions: ...

RECAP

- Math operators: `+ - * / % ( )`
- Math Library: `math.h`
  - Math Constants: `M_PI, M_E`
  - Elementary: `pow(b,e), sqrt(x), exp(x), log(x) , fmax(x,y), ...`
  - Tringonometric: `sin(x), asin(x), sinh(x), ...`
  - Special Functions: `lgamma(x), erf(x), ...`

WHAT'S THE OUTPUT?

```c
#include <stdio.h>


int main(void) {
    double a=1.345,b=1.123,c;
    c = a + b;
    printf("%.17lf+%.18lf=%.17lf",a,b,c); /* %.17lf interprets data as
double and prints number rounded off to 17 places after the decimal point.
*/
    return 0;
}
```

WHAT'S THE OUTPUT?

```c
#include <stdio.h>


int main(void) {
    double a=1.345,b=1.123,c;
    c = a + b;
    printf("%.17lf+%.18lf=%.17lf",a,b,c); /* %.17lf interprets data as
double and prints number rounded off to 17 places after the decimal point.
*/
    return 0;
}
```

1.34499999999999 7+1.12299999999999 98=2.46799999999999 7

IMPLICIT TYPE CONVERSION

- NUMBER CONSTANTS ARE `int` TYPE, RATIONAL CONSTANTS ARE `double` TYPE
- - IMPLICIT TYPE CONVERSION FROM TO
  -
- - IMPLICIT TYPE CONVERSION FROM TO

IMPLICIT TYPE CONVERSION

- NUMBER CONSTANTS ARE `int` TYPE, RATIONAL CONSTANTS ARE `double` TYPE
- `float a=3.4;` - IMPLICIT TYPE CONVERSION FROM `double` TO `float`
  -
- - IMPLICIT TYPE CONVERSION FROM TO

IMPLICIT TYPE CONVERSION

- NUMBER CONSTANTS ARE `int` TYPE, RATIONAL CONSTANTS ARE `double` TYPE
- `float a=3.4;` - IMPLICIT TYPE CONVERSION FROM `double` TO `float`
  - `float a=3.4f;`
- - IMPLICIT TYPE CONVERSION FROM TO

IMPLICIT TYPE CONVERSION

- NUMBER CONSTANTS ARE `int` TYPE, RATIONAL CONSTANTS ARE `double` TYPE
- `float a=3.4;` - IMPLICIT TYPE CONVERSION FROM `double` TO `float`
  - `float a=3.4f;`
- `fmax(3,4)` - IMPLICIT TYPE CONVERSION FROM `int` TO `double`

EXPLICIT TYPE CONVERSION

```
short int sum = 22, count = 5;
float mean = (float) sum / count; // count is implicitly converted!
```

Further reading: https://www.oreilly.com/library/view/c-in-a/0596006977/ch04.html

char DATATYPE

char DATATYPE

- `char grade = 'A';`
- `printf("Your grade is %c", grade);`
- `scanf("%c", &grade);`

char DATATYPE

- Represented as 7-bit code (leading bit of byte is 0)
- Can also be referred to using decimals 0-127

ASCII Table

0 0x00 0000 0000 NUL	1 0x01 0000 0001 SOH	2 0x02 0000 0010 STX	3 0x03 0000 0011 ETX	4 0x04 0000 0100 EOT	5 0x05 0000 0101 ENQ	6 0x06 0000 0110 ACK	7 0x07 0000 0111 BEL	8 0x08 0000 1000 BS	9 0x09 0000 1001 HT	10 0x0a 0000 1010 LF \n	11 0x0b 0000 1011 VT	12 0x0c 0000 1100 FF	13 0x0d 0000 1101 CR \r	14 0x0e 0000 1110 SO	15 0x0f 0000 1111 SI
16 0x10 0001 0000 DLE	17 0x11 0001 0001 DC1	18 0x12 0001 0010 DC2	19 0x13 0001 0011 DC3	20 0x14 0001 0100 DC4	21 0x15 0001 0101 NAK	22 0x16 0001 0110 SYN	23 0x17 0001 0111 ETB	24 0x18 0001 1000 CAN	25 0x19 0001 1001 EM	26 0x1a 0001 1010 SUB	27 0x1b 0001 1011 ESC	28 0x1c 0001 1100 FS	29 0x1d 0001 1101 GS	30 0x1e 0001 1110 RS	31 0x1f 0001 1111 US
32 0x20 0010 0000 SPACE	33 0x21 0010 0001 !	34 0x22 0010 0010 "	35 0x23 0010 0011 #	36 0x24 0010 0100 $	37 0x25 0010 0101 %	38 0x26 0010 0110 &	39 0x27 0010 0111 '	40 0x28 0010 1000 (	41 0x29 0010 1001)	42 0x2a 0010 1010 *	43 0x2b 0010 1011 +	44 0x2c 0010 1100 ,	45 0x2d 0010 1101 -	46 0x2e 0010 1110 .	47 0x2f 0010 1111 /
48 0x30 0011 0000 0	49 0x31 0011 0001 1	50 0x32 0011 0010 2	51 0x33 0011 0011 3	52 0x34 0011 0100 4	53 0x35 0011 0101 5	54 0x36 0011 0110 6	55 0x37 0011 0111 7	56 0x38 0011 1000 8	57 0x39 0011 1001 9	58 0x3a 0011 1010 :	59 0x3b 0011 1011 ;	60 0x3c 0011 1100 <	61 0x3d 0011 1101 =	62 0x3e 0011 1110 >	63 0x3f 0011 1111 ?
64 0x40 0100 0000 @	65 0x41 0100 0001 A	66 0x42 0100 0010 B	67 0x43 0100 0011 C	68 0x44 0100 0100 D	69 0x45 0100 0101 E	70 0x46 0100 0110 F	71 0x47 0100 0111 G	72 0x48 0100 1000 H	73 0x49 0100 1001 I	74 0x4a 0100 1010 J	75 0x4b 0100 1011 K	76 0x4c 0100 1100 L	77 0x4d 0100 1101 M	78 0x4e 0100 1110 N	79 0x4f 0100 1111 O
80 0x50 0101 0000 P	81 0x51 0101 0001 Q	82 0x52 0101 0010 R	83 0x53 0101 0011 S	84 0x54 0101 0100 T	85 0x55 0101 0101 U	86 0x56 0101 0110 V	87 0x57 0101 0111 W	88 0x58 0101 1000 X	89 0x59 0101 1001 Y	90 0x5a 0101 1010 Z	91 0x5b 0101 1011 [	92 0x5c 0101 1100 \	93 0x5d 0101 1101]	94 0x5e 0101 1110 ^	95 0x5f 0101 1111 _
96 0x60 0110 0000 `	97 0x61 0110 0001 a	98 0x62 0110 0010 b	99 0x63 0110 0011 c	100 0x64 0110 0100 d	101 0x65 0110 0101 e	102 0x66 0110 0110 f	103 0x67 0110 0111 g	104 0x68 0110 1000 h	105 0x69 0110 1001 i	106 0x6a 0110 1010 j	107 0x6b 0110 1011 k	108 0x6c 0110 1100 l	109 0x6d 0110 1101 m	110 0x6e 0110 1110 n	111 0x6f 0110 1111 o
112 0x70 0111 0000 p	113 0x71 0111 0001 q	114 0x72 0111 0010 r	115 0x73 0111 0011 s	116 0x74 0111 0100 t	117 0x75 0111 0101 u	118 0x76 0111 0110 v	119 0x77 0111 0111 w	120 0x78 0111 1000 x	121 0x79 0111 1001 y	122 0x7a 0111 1010 z	123 0x7b 0111 1011 {	124 0x7c 0111 1100 |	125 0x7d 0111 1101 }	126 0x7e 0111 1110 ~	127 0x7f 0111 1111 DEL

char DATATYPE

```
char letter = 'A'; // stores the number 65 in memory
printf("%c %d\n", letter, letter);
// Output: A 65
```

- USE ANY MATH OPERATOR
  - `grade = grade + 1;`

ctype.h

isalnum()	isalpha()	isblank()	iscntrl()	isdigit()
isgraph()	islower()	isprint()	ispunct()	isspace()
isupper()	isxdigit()	tolower()	toupper()	

LOGICAL OPERATORS, EXPRESSIONS

LOGICAL EQUALITY

- `number1==number2`
  - Evaluates to 0 if number1 is equal to number2
  - Evaluates to 1 if number1 is not equal to number2

LOGICAL EQUALITY

```
#include <stdio.h>
int main(void) {
    int number;
    printf("Enter a number:\n");
    scanf("%d",&number);
    printf("The statement: the number you entered is even is %u\n[0 means false; 1
means true].\n",




}
```

LOGICAL EQUALITY

```c
#include <stdio.h>
int main(void) {
    int number;
    printf("Enter a number:\n");
    scanf("%d",&number);
    printf("The statement: the number you entered is even is %u\n[0 means false; 1 means true].\n", (number%2)==0);
    // == is equality, evaluated to be 0(false)/1(true)
    // != is not-equal-to
    return 0;
}
```

WHAT'S THE OUTPUT?

```c
#include <stdio.h>
int main(void) {
    double a=1.345,b=1.123,c=a+b,d=a+10.234433e9+b-10.234433e9;
    printf("Statement %.3lf==%.3lf is: %u\n[0 means False, 1 means True]\n",c,d,c==d);
    return 0;
}
```

WHAT'S THE OUTPUT?

```c
#include <stdio.h>
int main(void) {
    double a=1.345,b=1.123,c=a+b,d=a+10.234433e9+b-10.234433e9;
    printf("Statement %.3lf==%.3lf is: %u\n[0 means False, 1 means True]\n",c,d,c==d);
    return 0;
}
```

Statement 2.468==2.468 is: 0
[0 means False, 1 means True]

c = 2.4679999999999997
d = 2.4679985046386718
c == d: 0

MATCHING RATIONALS

```c
#include <stdio.h>

int main(void) {
    double a=1.345,b=1.123,c=a+b,d=a+ 10.234433e9+b-10.234433e9;
    printf("Statement c==d is: %u\n",
```

Statement c==d is: 1

MATCHING RATIONALS

```c
#include <stdio.h>

int main(void) {
    double a=1.345,b=1.123,c=a+b,d=a+ 10.234433e9+b-10.234433e9;
    printf("Statement c==d is: %u\n", (c<=(d+1e-4))&&(c>=(d-1e-4)));
    // <= is less than or equal to. >= is greater than or equal to
    // && is logical AND;
    return 0;
}
```

```
Statement c==d is: 1
```

MATCHING RATIONALS

```c
#include <stdio.h>

int main(void) {
    double a=1.345,b=1.123,c=a+b,d=a+ 10.234433e9+b-10.234433e9;
    printf("Statement c==d is: %u\n", (c<=(d+1e-4))&&(c>=(d-1e-4)));
    // <= is less than or equal to. >= is greater than or equal to
    // && is logical AND; || is logical OR; ! is logical NOT
    return 0;
}
```

```
Statement c==d is: 1
```

EQUALITY AND RELATIONAL OPERATORS

Algebraic equality or relational operator	C equality or relational operator	Sample C condition	Meaning of C condition
Relational operators			
>	>	x > y	x is greater than y
<	<	x < y	x is less than y
≥	>=	x >= y	x is greater than or equal to y
≤	<=	x <= y	x is less than or equal to y
Equality operators			
=	==	x == y	x is equal to y
≠	!=	x != y	x is not equal to y

Operators				Grouping
()				left-to-right
*	/	%		left-to-right
+	-			left-to-right
<	<=	>	>=	left-to-right
==	!=			left-to-right
=				right-to-left

CONDITIONAL EXPRESSIONS

CONDITIONAL EXPRESSION

```c
#include <stdio.h>

int main(void) {
    double a=1.345,b=1.123,c=a+b,d=a+ 10.234433e9+b-10.234433e9;
    printf("Statement c==d is: %s\n", (c<=(d+1e-4))&&(c>=(d-1e-4))?"True":"False");
    // expr1 ? expr2 : expr3
    // %s is specifier for string
    return 0;
}
```

Statement c==d is: True

CONDITIONAL EXPRESSION

```c
#include <stdio.h>

int main(void) {
    int age;
    printf("Enter age in years\n");
    scanf("%d",&age);
    printf("Ticket cost is %d\n",(age>=12)?1000:500);
    return 0;
}
```

CONDITIONAL EXPRESSION

```c
#include <stdio.h>

int main(void) {
    int age;
    printf("Enter age in years\n");
    scanf("%d",&age);
    printf("Ticket cost is %d\n",(age>=12)?((age<=65)?1000:500):500);
    return 0;
}
```

CONDITIONAL EXPRESSION

```c
#include <stdio.h>

int main(void) {
    int age;
    printf("Enter age in years\n");
    scanf("%d",&age);
    printf("Ticket cost is %d\n",((age<12)||(age>65))?500:1000);
    return 0;
}
```

char DATATYPE

- USE ANY MATH/LOGIC OPERATOR
  - `grade = grade + 1;`
  - `printf("%c is %s", grade, ((grade <= 68) && (grade >= 'A')) ? "Valid grade" : "Invalid grade");`

YOUR OWN fmax

```
int student1_marks=90, student2_marks=100;
int max_marks= (student1_marks>=student2_marks) ? student1_marks : student2_marks;
```

3-WAY fmax

```
max_marks= (student1_marks>=student2_marks) ? ((student1_marks>=student3_marks) ?
student1_marks : student3_marks) : ((student2_marks>=student3_marks) ?
student2_marks : student3_marks);
```

3-WAY fmax

```
max_marks = ((student1_marks>=student2_marks)&&(student1_marks>=student3_marks)) ?
student1_marks :
(((student2_marks>=student1_marks)&&(student2_marks>=student3_marks)) ?
student2_marks : student3_marks);
```

3-WAY fmax

```
max_marks = (student1_marks>=student2_marks) ? student1_marks : student2_marks;
max_marks = (max_marks>=student3_marks) ? max_marks : student3_marks;
```

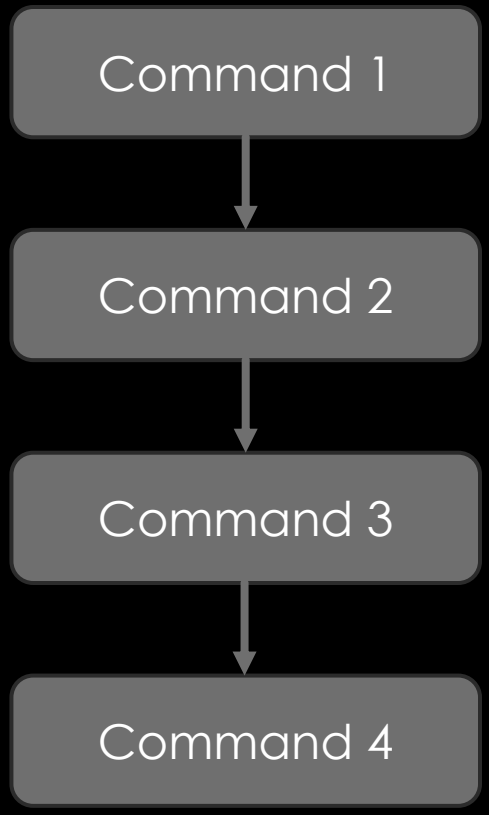
5-WAY fmax

```
max_marks = (student1_marks>=student2_marks) ? student1_marks : student2_marks;
max_marks = (max_marks>=student3_marks) ? max_marks : student3_marks;
max_marks = (max_marks>=student4_marks) ? max_marks : student4_marks;
max_marks = (max_marks>=student5_marks) ? max_marks : student5_marks;
```

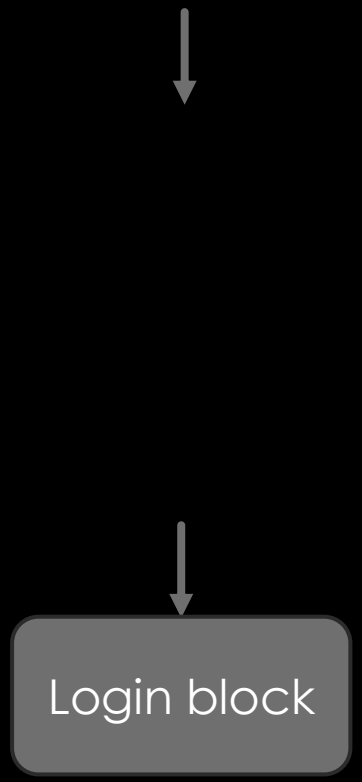
CONDITIONAL EXECUTION

IF-ELSE, SWITCH

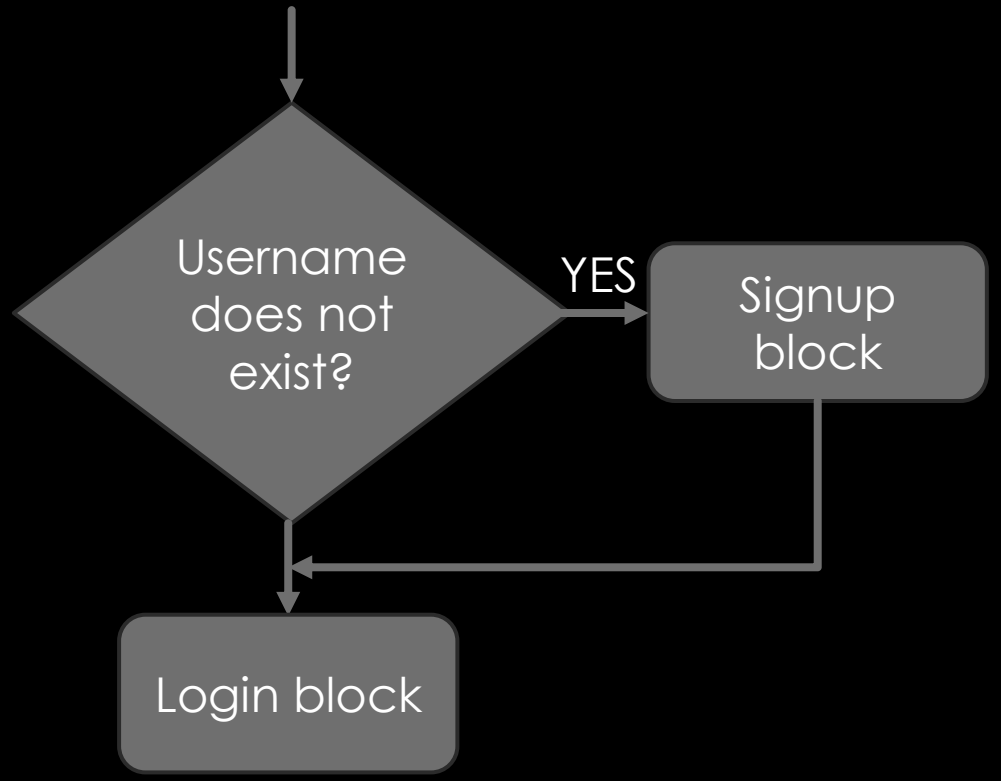
RECALL IMPERATIVE VIEW



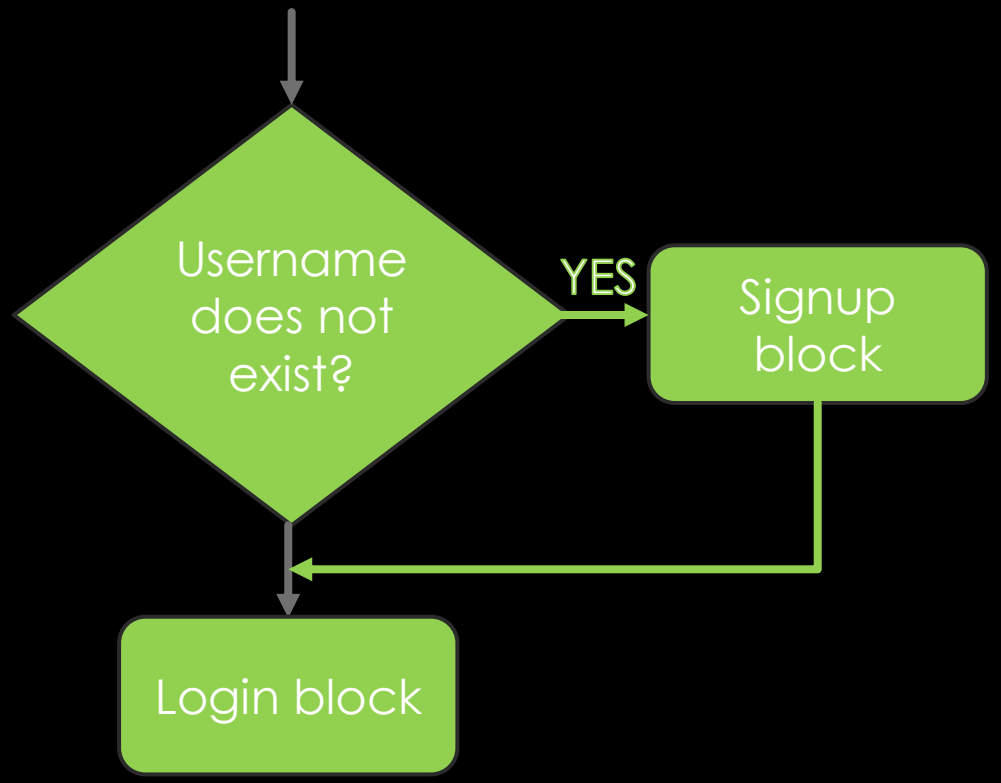
CONDITIONAL EXECUTION



CONDITIONAL EXECUTION



CONDITIONAL EXECUTION



IF STATEMENT SYNTAX

```
block-of-code

if(logical-expression){
\\ This block is executed if and only if the expression is TRUE
    block-of-code
}

block-of-code
```

CONDITIONAL EXECUTION: EXAMPLE

```c
#include <stdio.h>
int main(void) {
    int number; printf("Enter a number: "); scanf("%d",&number);\\Multiple commands

}
```

CONDITIONAL EXECUTION: EXAMPLE

```
#include <stdio.h>
int main(void) {
    int number; printf("Enter a number: "); scanf("%d",&number);
    if(number%2==0){\\Block executed if and only if number is divisible by 2
        printf("\nNumber you entered is an even number.\n");



    } \\Next command executed always

                                                    }
```

CONDITIONAL EXECUTION: EXAMPLE

```c
#include <stdio.h>
int main(void) {
    int number; printf("Enter a number: "); scanf("%d",&number);
    if(number%2==0){\\Block executed if and only if number is divisible by 2
        printf("\nNumber you entered is an even number.\n");



        return 0;\\If number is even, then program ends here
    } \\Next command executed if and only if number is not divisible by 2


                                                    }
```

CONDITIONAL EXECUTION: EXAMPLE

```
#include <stdio.h>
int main(void) {
    int number; printf("Enter a number: "); scanf("%d",&number);
    if(number%2==0){\\Block executed if and only if number is divisible by 2
        printf("\nNumber you entered is an even number.\n");



        return 0;\\If number is even, then program ends here
    } \\Next command executed if and only if number is divisible by 2
    printf("\nNumber you entered is an odd number.\n");
    return 0; \\If number is odd, then program ends here}
```

CONDITIONAL EXECUTION: EXAMPLE

```c
#include <stdio.h>
int main(void) {
    int number; printf("Enter a number: "); scanf("%d",&number);
    if(number%2==0){
        printf("\nNumber you entered is an even number.\n");
        if(log2(abs(number))==round(log2(abs(number)))){\\ Nested if
            printf("Number you entered is a power of two.\n");
        }
        return 0;
    }
    printf("\nNumber you entered is an odd number.\n");
    return 0;}
```

TIP: INDENTATION OF BLOCKS

```
#include <stdio.h>
int main(void) {
    int number; printf("Enter a number: "); scanf("%d",&number);
    if(number%2==0){
        printf("\nNumber you entered is an even number.\n");
        if(log2(abs(number))==round(log2(abs(number)))){
            printf("Number you entered is a power of two.\n");
        }
        return 0;
    }
    printf("\nNumber you entered is an odd number.\n");
    return 0;}
```

WHAT'S THE OUTPUT?

```
#include <stdio.h>
int main(void) {
    char false='0';
    if(0){
        printf("Statement 1 is printed\n");
    }
    if(false){
        printf("Statement 2 is printed\n");
    }
    return 0;
}
```

WHAT'S THE OUTPUT?

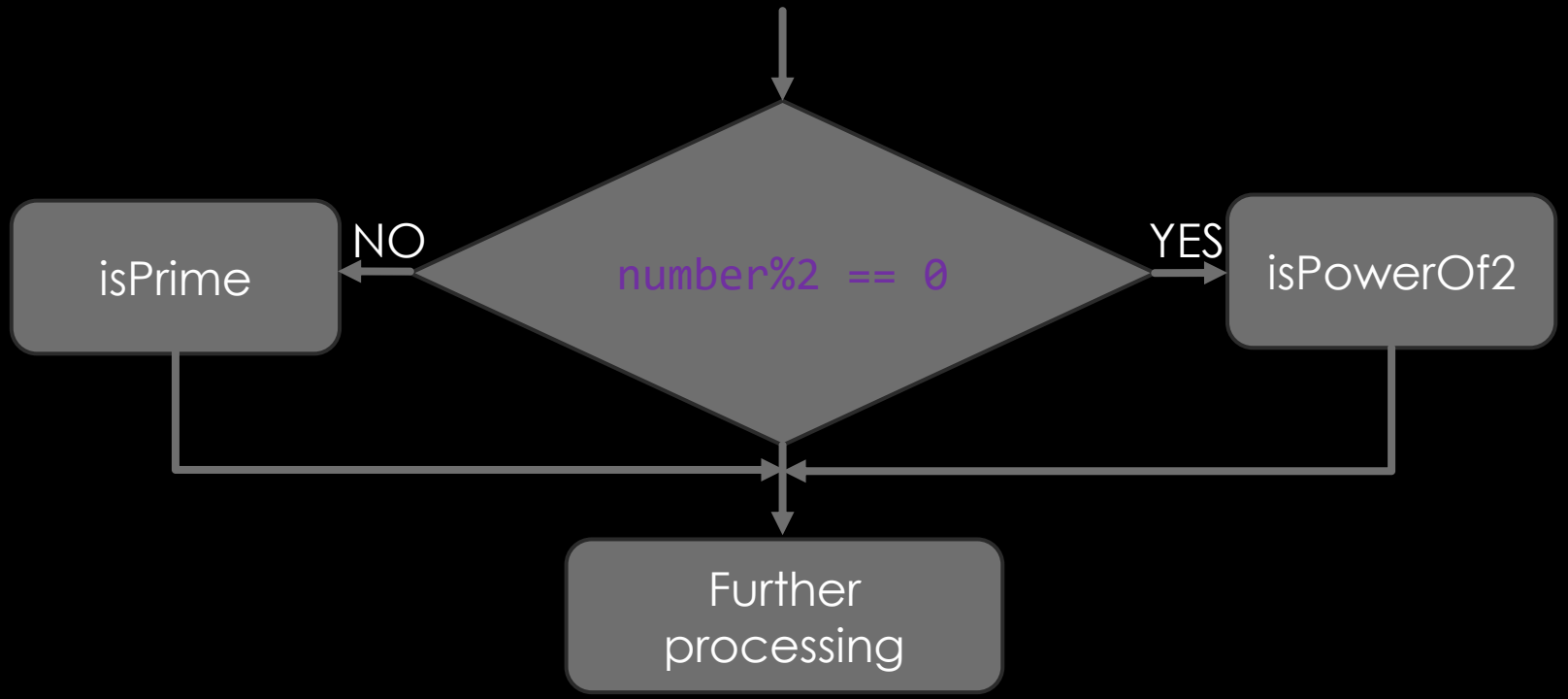
```c
#include <stdio.h>
int main(void) {
    char false='0';
    if(0){// 0 is FALSE, so this block is never executed
        printf("Statement 1 is printed\n");
    }
    if(false){// '0' value is 48, so this block is always executed
        printf("Statement 2 is printed\n");
    }
    return 0;
}
```

GENERALIZE ? : TO BLOCKS

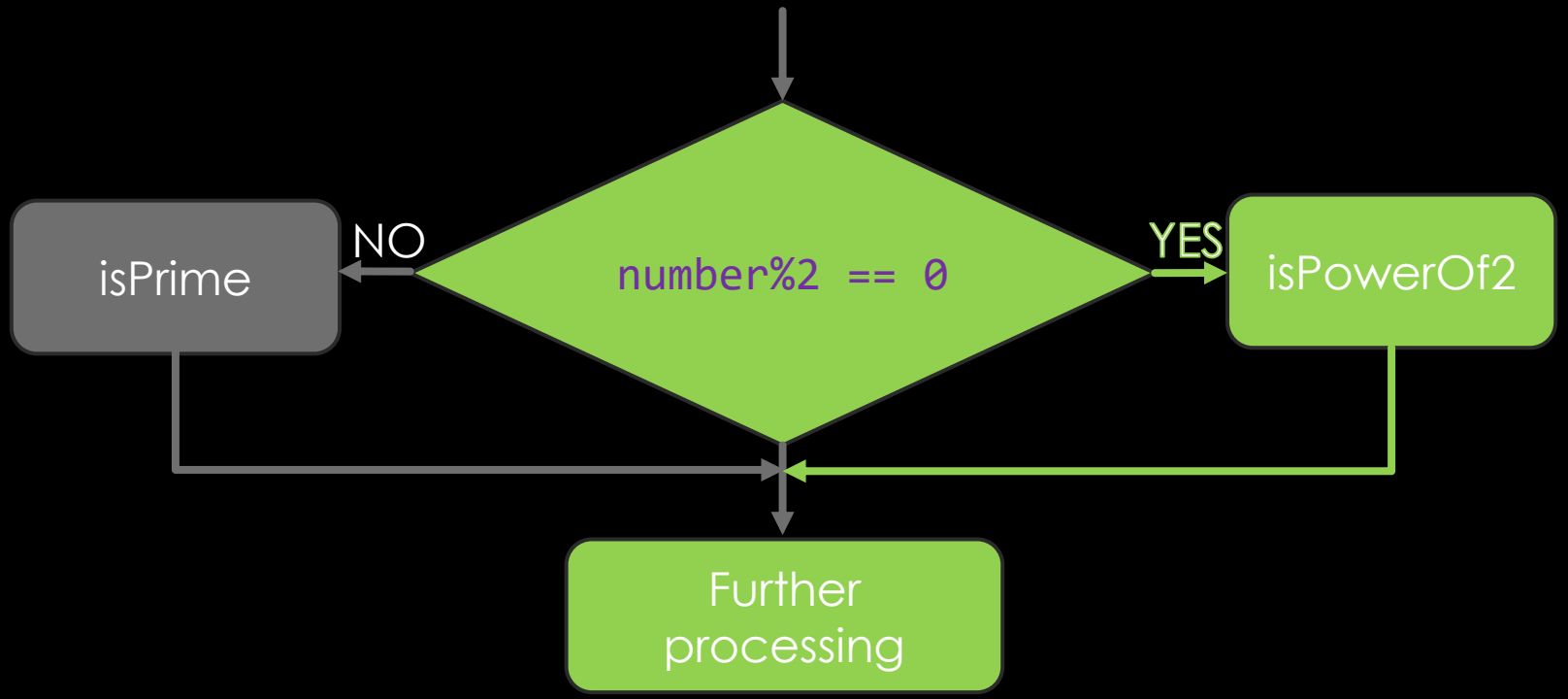
```
if(logical-expression){
// This block is executed if and only if the expression is TRUE
    block-of-code
}
else{
// This block is executed if and only if the expression is FALSE
    block-of-code
}

block-of-code
```

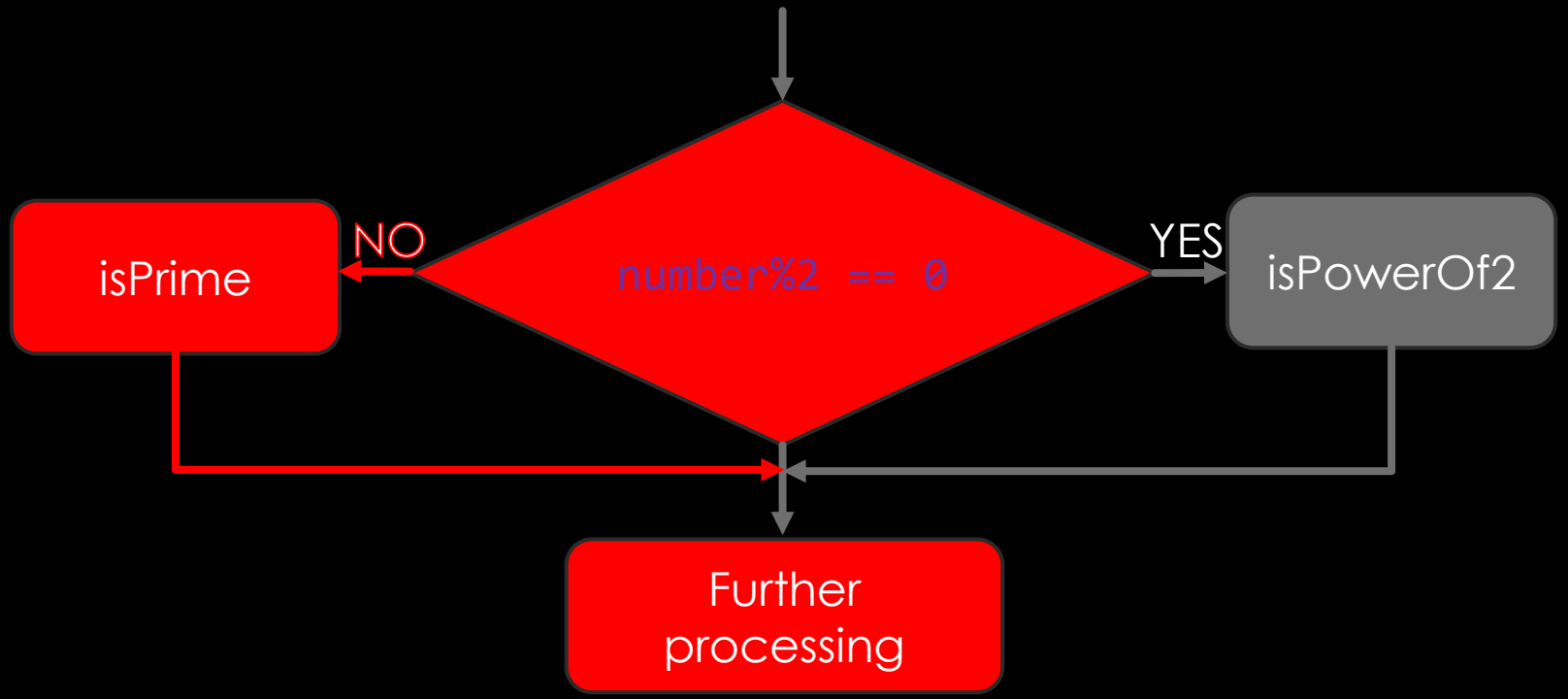
CONDITIONAL EXECUTION: IF-ELSE



CONDITIONAL EXECUTION: IF-ELSE



CONDITIONAL EXECUTION: IF-ELSE



CONDITIONAL EXECUTION: EXAMPLE

```c
#include <stdio.h>
int main(void) {
    int quantity; float price; printf("Enter no. tablets: "); scanf("%d", &quantity);
    if(quantity%10==0){//Block is executed if and only if number is divisible by 10
        printf("\nThanks for buying strips of tablets.\n");price=quantity*9;
    }
}
```

CONDITIONAL EXECUTION: EXAMPLE

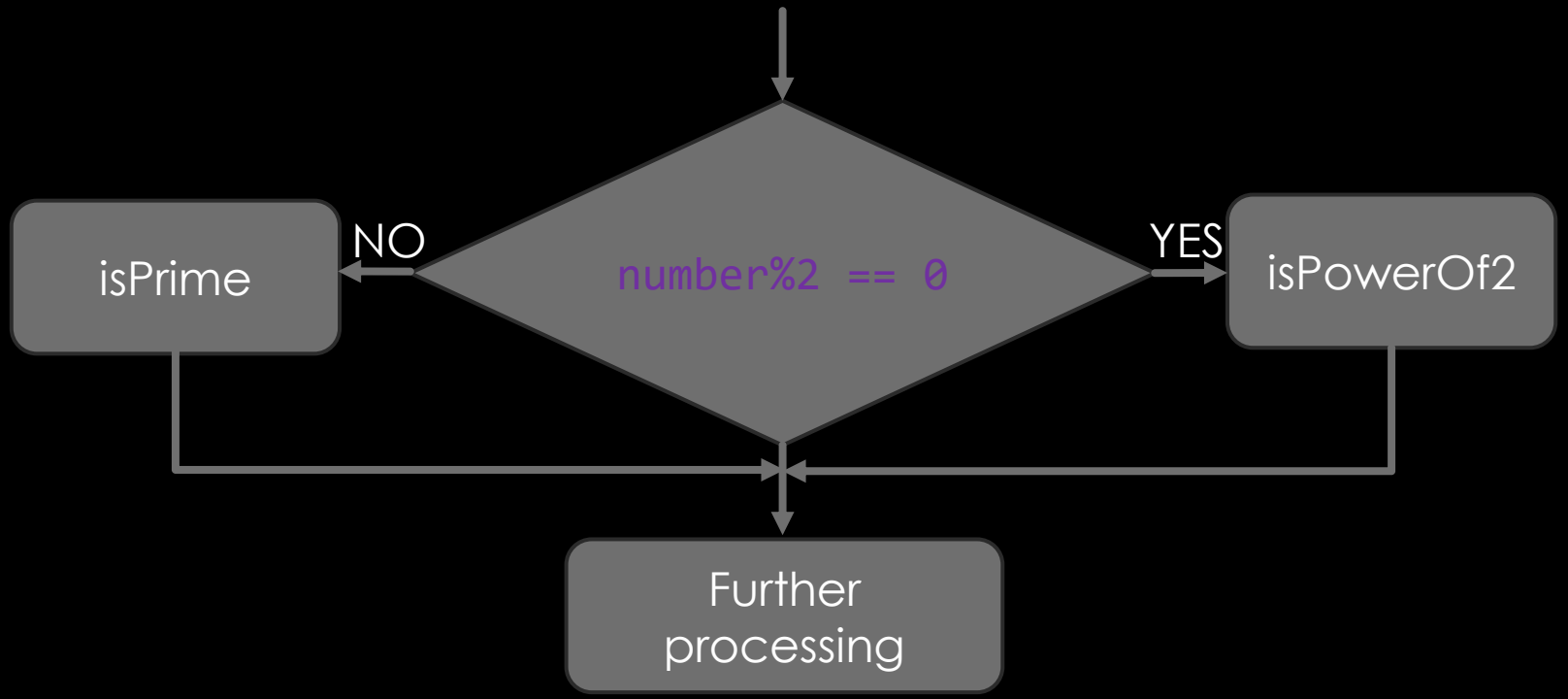
```c
#include <stdio.h>
int main(void) {
    int quantity; float price; printf("Enter no. tablets: "); scanf("%d", &quantity);
    if(quantity%10==0){//Block is executed if and only if number is divisible by 10
        printf("\nThanks for buying strips of tablets.\n");price=quantity*9;
    }
    else{// This block is executed if and only if number is not divisible by 10
        printf("\nWe will cut strips for you! No problem.\n");
        price=(quantity/10*90)+(quantity%10)*12;
    }

                                                    }
```

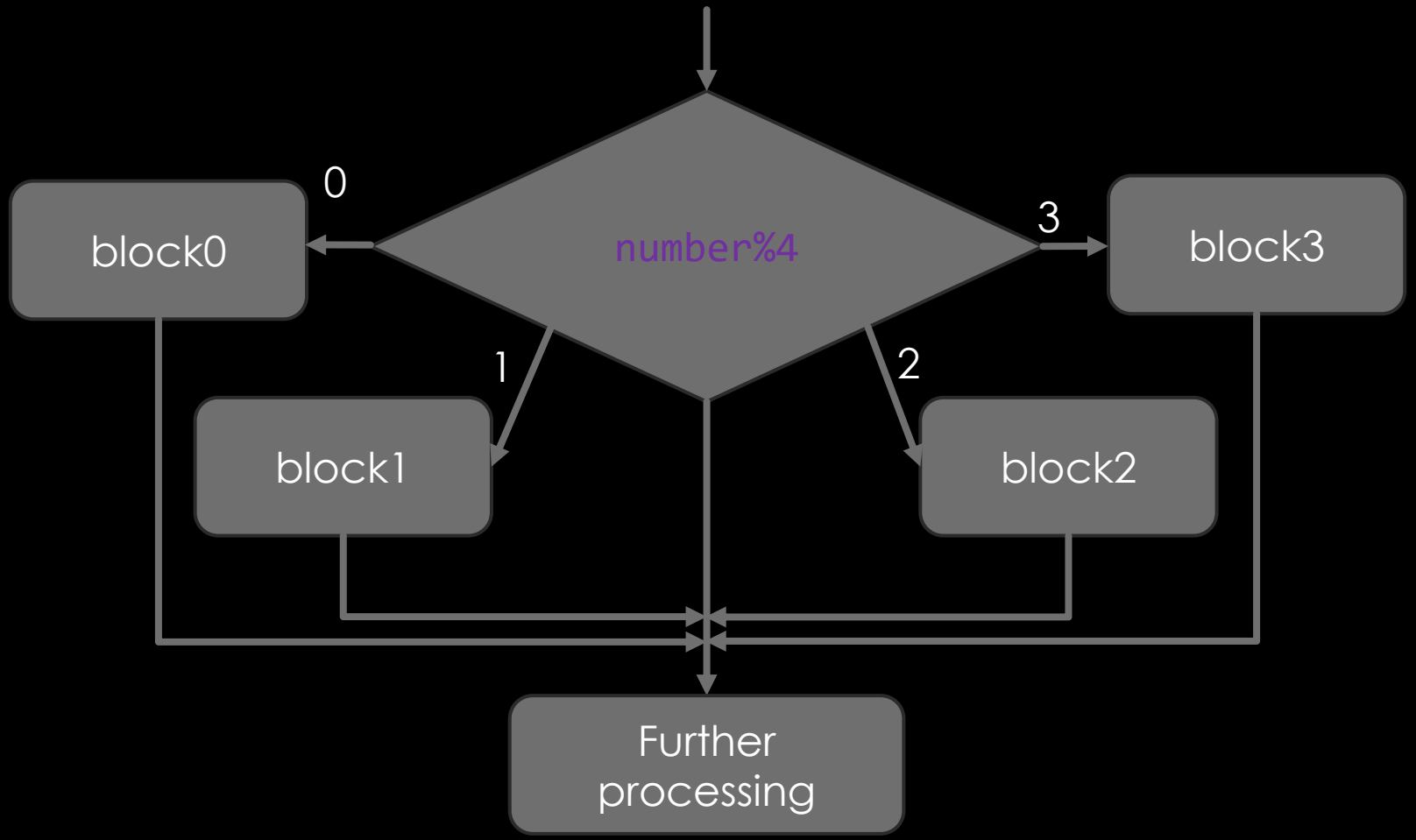
CONDITIONAL EXECUTION: EXAMPLE

```c
#include <stdio.h>
int main(void) {
    int quantity; float price; printf("Enter no. tablets: "); scanf("%d", &quantity);
    if(quantity%10==0){//Block is executed if and only if number is divisible by 10
        printf("\nThanks for buying strips of tablets.\n");price=quantity*9;
    }
    else{// This block is executed if and only if number is not divisible by 10
        printf("\nWe will cut strips for you! No problem.\n");
        price=(quantity/10*90)+(quantity%10)*12;
    }
    price=price*1.1;// This and next are always executed
    printf("Please pay Rs.%.2f/- only.", price); return 0;}
```

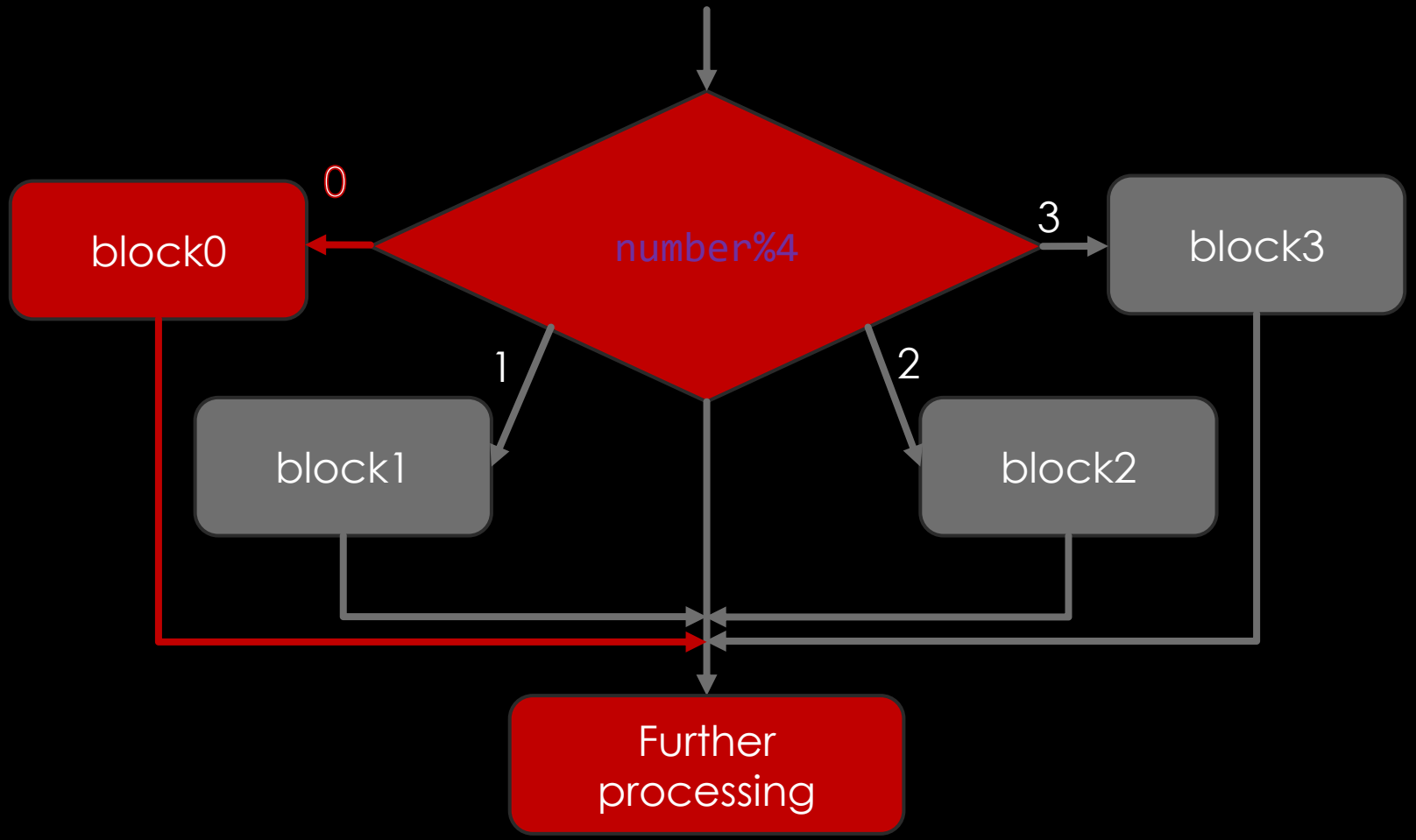
CONDITIONAL EXECUTION: IF-ELSE



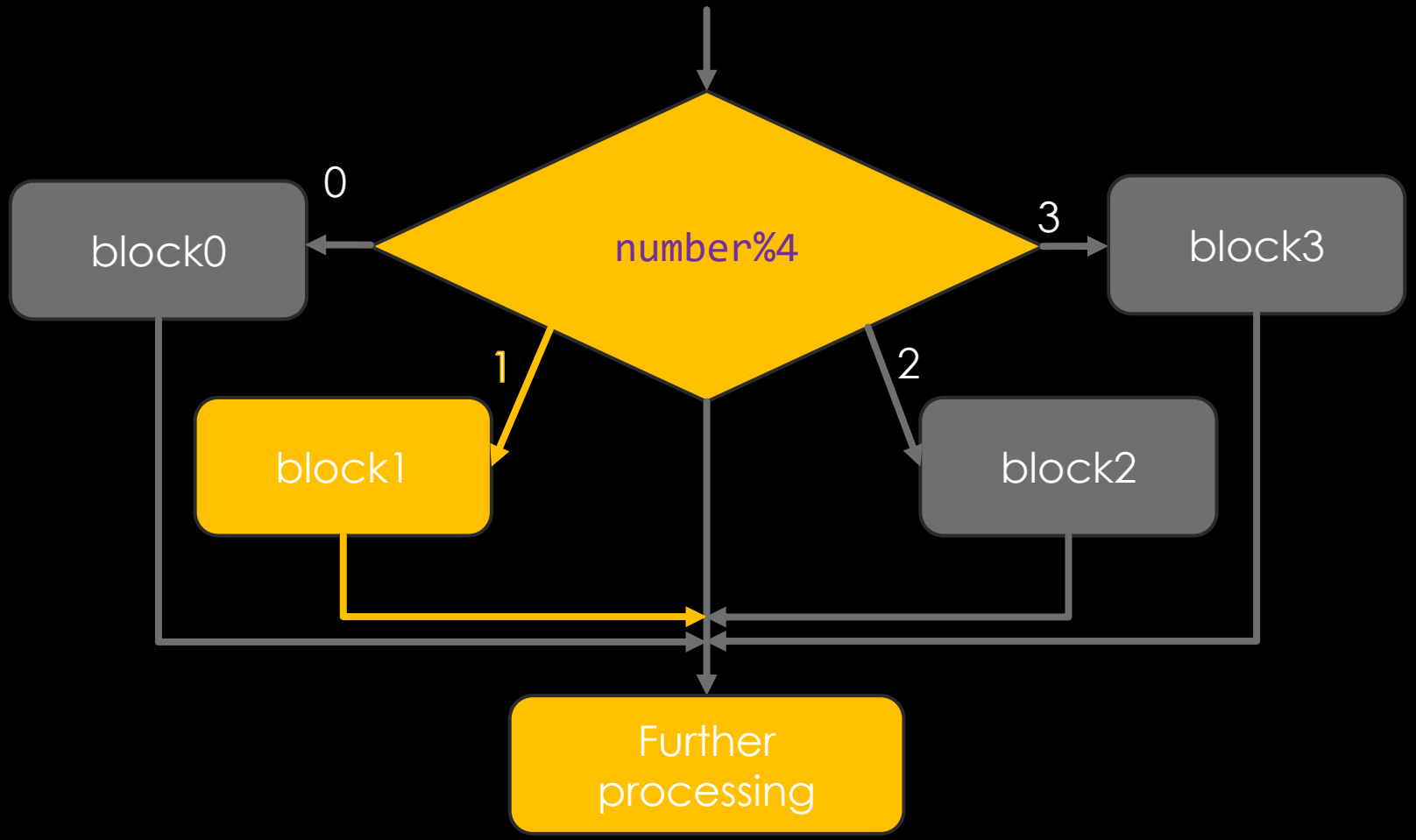
CONDITIONAL EXECUTION: SWITCH



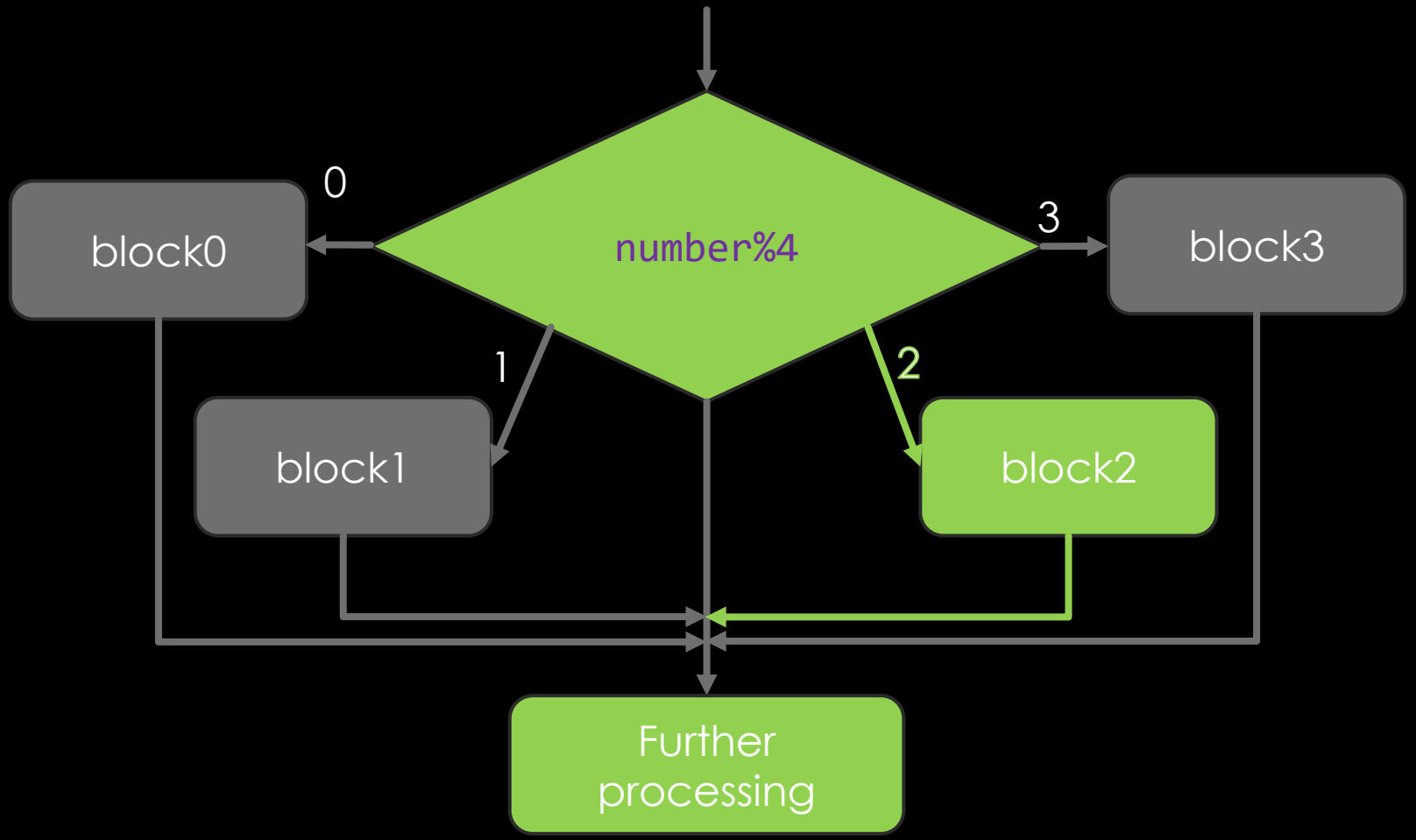
CONDITIONAL EXECUTION: SWITCH



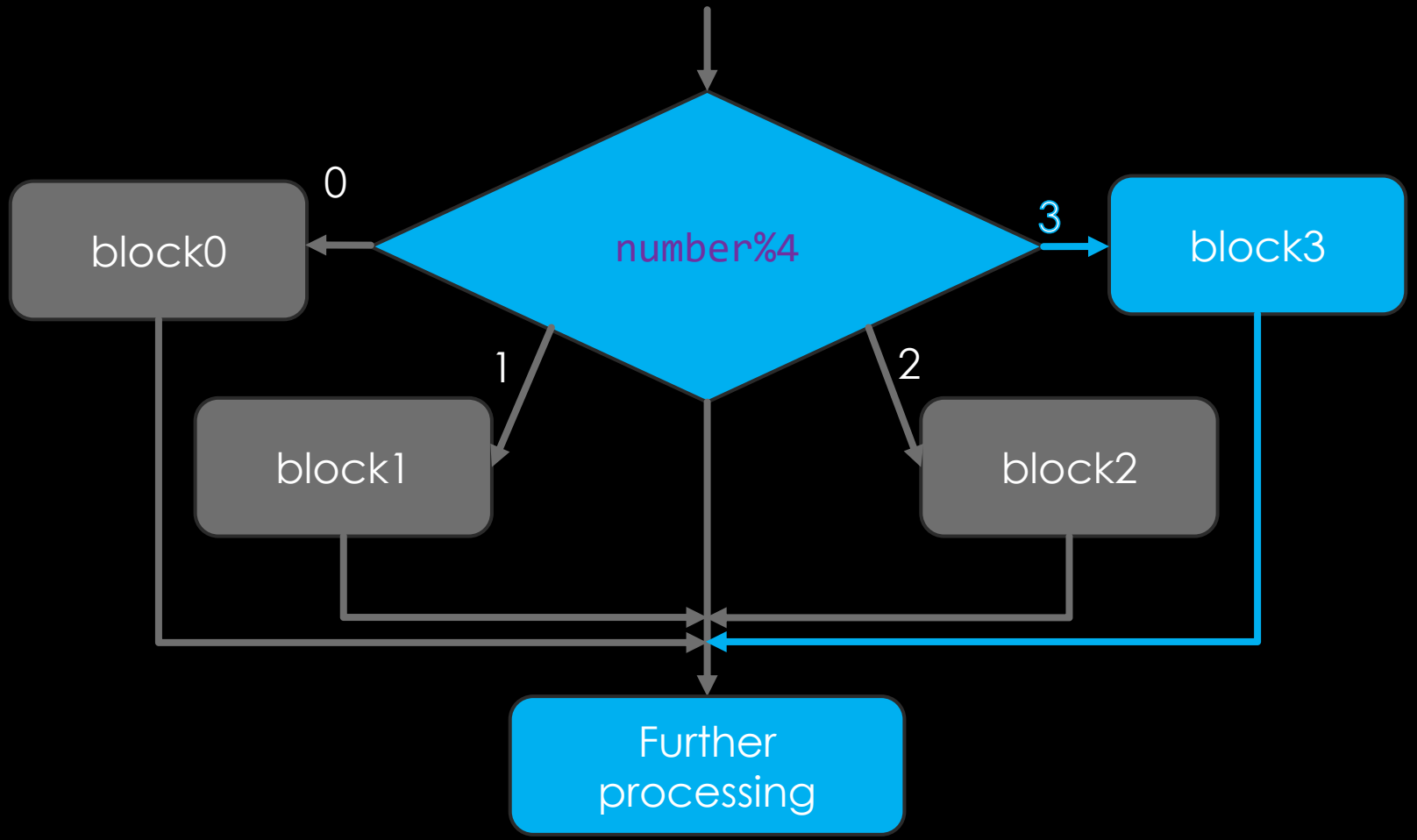
CONDITIONAL EXECUTION: SWITCH



CONDITIONAL EXECUTION: SWITCH



CONDITIONAL EXECUTION: SWITCH

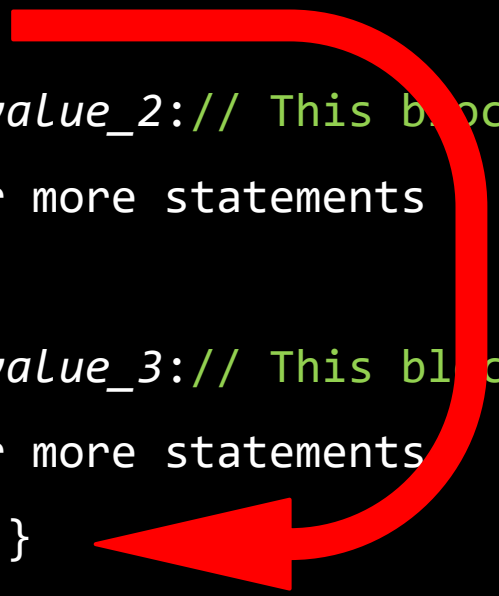


switch SYNTAX

```
block-of-code
switch(expression){
    case value_1:// This block is executed if and only if the expression==value_1
    one or more statements
    break;
    case value_2:// This block is executed if and only if the expression==value_2
    one or more statements
    break;
    case value_3:// This block is executed if and only if the expression==value_3
    one or more statements
    break;}
block-of-code
```

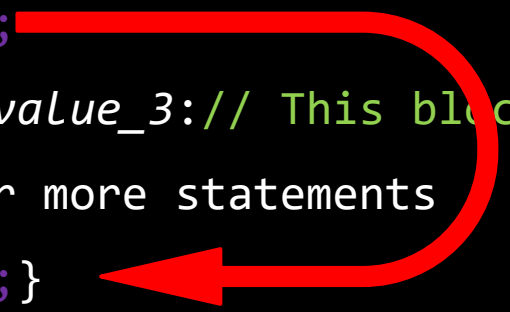
switch SYNTAX

```
block-of-code
switch(expression){
    case value_1:// This block is executed if and only if the expression==value_1
    one or more statements
    break;
    case value_2:// This block is executed if and only if the expression==value_2
    one or more statements
    break;
    case value_3:// This block is executed if and only if the expression==value_3
    one or more statements
    break;}
block-of-code
```



switch SYNTAX

```
block-of-code
switch(expression){
    case value_1:// This block is executed if and only if the expression==value_1
    one or more statements
    break;
    case value_2:// This block is executed if and only if the expression==value_2
    one or more statements
    break;
    case value_3:// This block is executed if and only if the expression==value_3
    one or more statements
    break;}
block-of-code
```



CALCULATOR PROGRAM

```c
#include <stdio.h>
int main(void) {
    char operator; double number1, number2; printf("Enter [number][operator][number]\n");
    double result; scanf("%lf%c%lf", &number1,&operator,&number2);









}
```

CALCULATOR PROGRAM

```c
#include <stdio.h>
int main(void) {
    char operator; double number1, number2; printf("Enter [number][operator][number]\n");
    double result; scanf("%lf%c%lf", &number1,&operator,&number2);
    switch(operator){
        case '+':  result=number1+number2;break;
        case '-':  result=number1-number2;break;
        case '*':  result=number1*number2;break;
        case '/':  result=number1/number2;break;

    }
    printf("=%lf\n",result);return 0;}
```

CALCULATOR PROGRAM

```c
#include <stdio.h>
int main(void) {
    char operator; double number1, number2; printf("Enter [number][operator][number]\n");
    double result; scanf("%lf%c%lf", &number1,&operator,&number2);
    switch(operator){
        case '+':  result=number1+number2;break;
        case '-':  result=number1-number2;break;
        case '*':  result=number1*number2;break;
        case '/':  result=number1/number2;break;
        default :  printf("Invalid Operator\n");return 0;
    }
    printf("=%lf\n",result);return 0;}
```

BEHIND THE SCENES: SWITCH

- All case's commands are pooled into one block
- true case is used to index into the correct starting command
- Hence execution will continue sequentially until a `break/return`

BEHIND THE SCENES: SWITCH

- ALL CASE'S COMMANDS ARE POOLED INTO ONE BLOCK
- TRUE CASE IS USED TO INDEX INTO THE CORRECT STARTING COMMAND
- HENCE EXECUTION WILL CONTINUE SEQUENTIALLY UNTIL A `break/return`

In if-else, compiler adds break commands automatically

WHY NOT ADD `break` AUTOMATICALLY IN SWITCH?

WHY NOT ADD `break` AUTOMATICALLY IN SWITCH?

```
switch(expression){
    case value_1:
    case value_2:
    case value_3:
    one or more statements; break;
    case value_4:
    one or more statements; break;
    case value_5:
    one or more statements; break;
}
```

fall-through (executing multiple case blocks sequentially) is sometimes intentional and useful.

```
switch (ch) {
   case 'a':
   case 'A':
      printf("You entered A\n");
      break;
}
```

FEW SHORTHANDS

- a=a+2 can be written as a+=2
  - Similarly -= *= /=
-
  -

FEW SHORTHANDS

- a=a+2 can be written as a+=2
  - Similarly -= *= /=
- a=a+1 can be written as a++ or as ++a
  - Similarly a-- --a

POST AND PRE INCREMENT

- `a++` uses a in the expression and after the read increments it.
  - E.g., `c=a-(b++);` is same as `c=a-b;b+=1;`
- `++a` increments a first and then uses it in the command.
  - E.g., `c=a-(++b);` is same as `b+=1;c=a-b;`

THINK!

- `a=2;a=(a++)+(++a);`
-

THINK!

- `a=2;a=(a++)+(++a); // a=6`
-

THINK!

- `a=2;a=(a++)+(++a); // a=6`
- `a=2;a=(++a)+(++a);`

THINK!

- `a=2;a=(a++)+(++a); // a=6`
- `a=2;a=(++a)+(++a); // a=7`